task05_report

Author: Apurv Kumar
Project: Design Subnet for a Small Network
Date: Wed 25, 2026

Subnets

A subnet is a logical subdivision of an IP network. The act of splitting a large network into smaller networks is known as subnetting.

Subnetting can be performed using two methods:

1. **Fixed Length Subnet Masking (FLSM):**
   In FLSM, all subnets are equal in size and contain the same number of hosts. This approach can lead to IP address wastage because each subnet is allocated the same capacity regardless of actual need.

2. **Variable Length Subnet Masking (VLSM):**
   In VLSM, subnet sizes can vary, allowing each subnet to support a different number of hosts. This makes IP address allocation more efficient, as subnet masks are assigned based on specific requirements, resulting in minimal address wastage.

Additional details about subnets can be found in the referenced material.

The program **subnet.py** implements both subnetting techniques.

For Fixed Length Subnet Masking, the program accepts an IP address and a number *n* (where the limit of *n* depends on the IP address class). It divides the network into 2^n subnets, meaning the total number of subnets must be a power of 2. The minimum subnet size is set to 4 addresses (Subnet ID, Broadcast ID, and 2 usable host addresses). Since all subnets are equal in size, they share the same subnet mask. The program outputs the subnet mask along with the Subnet ID and Broadcast ID for each subnet.

Example run:

```
Enter the IP address
12.56.71.22

The IP Address belongs to : Class A

Select the type of Subnetting : '1' for FIXED and '2' for VARIABLE : 1

Enter the number 'n' such that network is divided in 2^n subnets and n is
in range(0, 22)
2

The number of subnets are : 4

Subnet Mask is :
255.192.0.0
```

```
The Network and Broadcast IDs for all subnets are :
1) SubnetID   : 12.0.0.0    BroadcastID : 12.63.255.255
2) SubnetID   : 12.64.0.0   BroadcastID : 12.127.255.255
3) SubnetID   : 12.128.0.0  BroadcastID : 12.191.255.255
4) SubnetID   : 12.192.0.0  BroadcastID : 12.255.255.255
```

For variable length subnet masking, the program takes input an IP address and the number of subnets user wants. Here, the number of subnets need not be a power of 2. Further, the program also ask user to input the number of hosts required in each subnet. The output of the program is (Subnet-mask, Subnet-ID, Broadcast-ID) for each subnet.

Example run:

```
Enter the IP address
104.96.34.59

The IP Address belongs to : Class A

Select the type of Subnetting : '1' for FIXED and '2' for VARIABLE : 2

Enter the number of subnets in the range(0, 4194304)
3

Enter the number of hosts in each subnet :
100000
324
62

Subnet Mask, NetworkID and BroadcastID for all the subnets are as follows :

1) SubnetMask : 255.254.0.0     SubnetID : 104.0.0.0      BroadcastID :
104.1.255.255
2) SubnetMask : 255.255.254.0   SubnetID : 104.2.0.0      BroadcastID :
104.2.1.255
3) SubnetMask : 255.255.255.192 SubnetID : 104.2.2.0      BroadcastID :
104.2.2.63
```

Usage :

```
python3 subnet.py
```

```
import math

class IP():
    def __init__(self, address):
        self.address = address
```

```
        self.subnet_mask = 0
        self.ip_class = ""
        self.no_of_subnets = 0
        self.subnet_power = 0

    def NetworkClass(self):
        x = self.address >> 24
        if 0 <= x <= 127 :
            print("The IP Address belongs to : Class A")
            self.ip_class = "Class A"
        elif 128 <= x <= 191 :
            print("The IP Address belongs to : Class B")
            self.ip_class = "Class B"
        elif 192 <= x <= 223 :
            print("The IP Address belongs to : Class C")
            self.ip_class = "Class C"
        elif 224 <= x <= 239 :
            print("The IP Address belongs to : Class D")
            self.ip_class = "Class D"
            exit("Subnetting is not possible")
        elif 240 <= x <= 255 :
            print("The IP Address belongs to : Class E")
            self.ip_class = "Class E"
            exit("Subnetting is not possible")
        else :
            exit("Invalid IP address")

    def FixedSubnets(self):
        if self.ip_class == 'Class A' :
            print("\nEnter the number 'n' such that network is divided in
2^n subnets and n is in range(%d, %d)" % (0, 22))
            self.subnet_power = int(input())
            if self.subnet_power > 22 :
                exit("Error !!! Follow the range")
            self.no_of_subnets = 2**self.subnet_power

            print("\nThe number of subnets are : %d" % self.no_of_subnets)

            self.subnet_mask = 0xFF000000 | (~(2 ** (24-self.subnet_power)
- 1))
            print("\nSubnet Mask is :")
            PrintIP(self.subnet_mask)

            size = int((2**24) / self.no_of_subnets)

            print("\nThe Network and Broadcast IDs for all subnets are :")
            networkid = self.address & 0xFF000000
            for i in range(0, self.no_of_subnets) :
                x = (networkid + (i * size))
                y = (networkid + ((i + 1) * size)) - 1
                print("%d) \tSubnetID   : " % (i + 1), end = '')
                PrintIP(x)
                print("\tBroadcastID : ", end = '')
                PrintIP(y)
```

```
                print("")

        elif self.ip_class == 'Class B' :
            print("\nEnter the number 'n' such that network is divided in
2^n subnets and n is in range(%d, %d)" % (0, 14))
            self.subnet_power = int(input())
            if self.subnet_power > 14 :
                exit("Error !!! Follow the range")
            self.no_of_subnets = 2**self.subnet_power

            print("\nThe number of subnets are : %d" % self.no_of_subnets)

            self.subnet_mask = 0xFFFF0000 | (~(2 ** (16-self.subnet_power)
- 1))
            print("\nSubnet Mask is :")
            PrintIP(self.subnet_mask)

            size = int((2**16) / self.no_of_subnets)

            print("\nThe Network and Broadcast IDs for all subnets are :")
            networkid = self.address & 0xFFFF0000
            for i in range(0, self.no_of_subnets) :
                x = (networkid + (i * size))
                y = (networkid + ((i + 1) * size)) - 1
                print("%d) \tSubnetID   : " % (i + 1), end = '')
                PrintIP(x)
                print("\tBroadcastID : ", end = '')
                PrintIP(y)
                print("")

        elif self.ip_class == 'Class C' :
            print("\nEnter the number 'n' such that network is divided in
2^n subnets and n is in range(%d, %d)" % (0, 6))
            self.subnet_power = int(input())
            if self.subnet_power > 6 :
                exit("Error !!! Follow the range")
            self.no_of_subnets = 2**self.subnet_power

            print("\nThe number of subnets are : %d" % self.no_of_subnets)

            self.subnet_mask = 0xFFFFFF00 | (~(2 ** (8-self.subnet_power) -
1))
            print("\nSubnet Mask is :")
            PrintIP(self.subnet_mask)

            size = int((2**8) / self.no_of_subnets)

            print("\nThe Network and Broadcast IDs for all subnets are :")
            networkid = self.address & 0xFFFFFF00
            for i in range(0, self.no_of_subnets) :
                x = (networkid + (i * size))
                y = (networkid + ((i + 1) * size)) - 1
                print("%d) \tSubnetID   : " % (i + 1), end = '')
                PrintIP(x)
```

```
                print("\tBroadcastID : ", end = '')
                PrintIP(y)
                print("")

    def VariableSubnets(self) :
        if self.ip_class == 'Class A' :
            print("\nEnter the number of subnets in the range(%d, %d)" %
(0, 2**22))
            self.no_of_subnets = int(input())
            print("")
            if self.no_of_subnets > 2**22 :
                exit("Error !!! Follow the range")

            subnet_sizes = list()
            print("Enter the number of hosts in each subnet :")
            for i in range(0, self.no_of_subnets) :
                subnet_sizes.append(int(input()) + 2)
                pos = math.ceil(math.log(subnet_sizes[i], 2))
                subnet_sizes[i] = 2**pos

            print("")
            if sum(subnet_sizes) > (2**24) :
                exit("Subnets cannot accomodate required number hosts")

            print("Subnet Mask, NetworkID and BroadcastID for all the
subnets are as follows :")
            networkid = self.address & 0xFF000000
            for i in range(0, self.no_of_subnets) :
                self.subnet_mask = 0xFF000000 | (~(subnet_sizes[i] - 1))
                print("%d) \tSubnetMask :" % (i+1), end = " ")
                PrintIP(self.subnet_mask)
                print("\tSubnetID :", end = " ")
                PrintIP(networkid)
                networkid = networkid + subnet_sizes[i]
                print("\tBroadcastID :", end = " ")
                PrintIP(networkid-1)
                print("")

        elif self.ip_class == 'Class B' :
            print("\nEnter the number of subnets in the range(%d, %d)" %
(0, 2**14))
            self.no_of_subnets = int(input())
            print("")
            if self.no_of_subnets > 2**16 :
                exit("Error !!! Follow the range")

            subnet_sizes = list()
            print("Enter the number of hosts in each subnet :")
            for i in range(0, self.no_of_subnets) :
                subnet_sizes.append(int(input()) + 2)
                pos = math.ceil(math.log(subnet_sizes[i], 2))
                subnet_sizes[i] = 2**pos

            print("")
```

```
            if sum(subnet_sizes) > (2**16) :
                exit("Subnets cannot accomodate required number hosts")

            print("Subnet Mask, NetworkID and BroadcastID for all the
subnets are as follows :")
            networkid = self.address & 0xFFFF0000
            for i in range(0, self.no_of_subnets) :
                self.subnet_mask = 0xFFFF0000 | (~(subnet_sizes[i] - 1))
                print("%d) \tSubnetMask :" % (i+1), end = " ")
                PrintIP(self.subnet_mask)
                print("\tSubnetID :", end = " ")
                PrintIP(networkid)
                networkid = networkid + subnet_sizes[i]
                print("\tBroadcastID :", end = " ")
                PrintIP(networkid-1)
                print("")

        elif self.ip_class == 'Class C' :
            print("\nEnter the number of subnets in the range(%d, %d)" %
(0, 2**6))
            self.no_of_subnets = int(input())
            print("")
            if self.no_of_subnets > 2**6 :
                exit("Error !!! Follow the range")

            subnet_sizes = list()
            print("Enter the number of hosts in each subnet :")
            for i in range(0, self.no_of_subnets) :
                subnet_sizes.append(int(input()) + 2)
                pos = math.ceil(math.log(subnet_sizes[i], 2))
                subnet_sizes[i] = 2**pos

            print("")
            if sum(subnet_sizes) > (2**8) :
                exit("Subnets cannot accomodate required number hosts")

            print("Subnet Mask, NetworkID and BroadcastID for all the
subnets are as follows :")
            networkid = self.address & 0xFFFFFF00
            for i in range(0, self.no_of_subnets) :
                self.subnet_mask = 0xFFFFFF00 | (~(subnet_sizes[i] - 1))
                print("%d) \tSubnetMask :" % (i+1), end = " ")
                PrintIP(self.subnet_mask)
                print("\tSubnetID :", end = " ")
                PrintIP(networkid)
                networkid = networkid + subnet_sizes[i]
                print("\tBroadcastID :", end = " ")
                PrintIP(networkid-1)
                print("")

def PrintIP(address) :
    print((address & 0xFF000000) >> 24, (address & 0xFF0000) >> 16,
(address & 0xFF00) >> 8, address & 0xFF, sep = ".")
```

```
if __name__ == "__main__" :
    print("Enter the IP address")
    try :
        temp = list(map(int, input().split(".")))
    except :
        exit("Error")

    if len(temp) != 4 :
        exit("Invalid IP Address")

    address = 0
    address = temp[0] << 24
    address = address + (temp[1] << 16)
    address = address + (temp[2] << 8)
    address = address + temp[3]
    ip = IP(address)
    ip.NetworkClass()
    print("\nSelect the type of Subnetting :\n'1' for FIXED and '2' for
VARIABLE :")
    typ = int(input())
    if typ == 1:
        ip.FixedSubnets()
    elif typ == 2 :
        ip.VariableSubnets()
```

Classless Inter-Domain Routing (CIDR)

Classless Inter-Domain Routing (CIDR) is a technique used for IP address allocation and routing. It is based on Variable Length Subnet Masking (VLSM), which enables the use of prefixes with flexible lengths. CIDR introduced a new way to represent IP addresses, known as CIDR notation, where an IP address or routing prefix is followed by a suffix that specifies the number of prefix bits, for example, 192.4.5.12/24. For more details, refer to the provided link.

The program **cidr.py** implements CIDR functionality. It first accepts an IP address written in CIDR notation and displays all relevant details associated with that address, including the maximum number of subnets, the maximum number of possible addresses, the address range, network ID, broadcast address, and subnet mask.

After this, the user is prompted to enter the desired number of subnets and the required number of hosts for each subnet. The program then divides the network into subnets with minimal IP address wastage and outputs all the necessary information for each subnet.

Example run:

```
Enter the IP address in '/' notation
192.0.0.0/8

Consider Minimum size of a single subnet as 4 address :
Maximum number of subnets possible = 4194304
Maximum number of address = 16777216
```

```
Address Range :
Start Address = 192.0.0.0/8
End Address = 192.255.255.255/8
Network ID = 192.0.0.0/8
Broadcast Address = 192.255.255.255/8
Network Mask = 255.0.0.0/8

Enter the number of subnets in the range(0, 4194304)
4

Enter the number of hosts in each subnet :
12
12
14
14

Subnet Mask, SubnetID and BroadcastID for all the subnets are as follows :

1) SubnetMask : 255.255.255.240 SubnetID : 192.0.0.0/28 BroadcastID :
192.0.0.15/28 MaskBits : 28
2) SubnetMask : 255.255.255.240 SubnetID : 192.0.0.16/28 BroadcastID :
192.0.0.31/28 MaskBits : 28
3) SubnetMask : 255.255.255.240 SubnetID : 192.0.0.32/28 BroadcastID :
192.0.0.47/28 MaskBits : 28
4) SubnetMask : 255.255.255.240 SubnetID : 192.0.0.48/28 BroadcastID :
192.0.0.63/28 MaskBits : 28
```

Usage :

```
python3 cidr.py
```

```python
import math

class IP():
    def __init__(self, address, hostbits):
        self.address = address
        self.hostbits = hostbits
        self.network_mask = 0
        self.max_no_of_subnets = 0
        self.max_no_of_address = 0
        self.start_address = 0
        self.end_address = 0
        self.no_of_subnets = 0

    def Info(self):
        print("\nConsider Minimum size of a single subnet as 4 address :")
        self.max_no_of_subnets = (2 ** (self.hostbits - 2))
        print("Maximum number of subnets possible = %d" %
self.max_no_of_subnets)
```

```
        self.max_no_of_address = (2 ** self.hostbits)
        print("Maximum number of address = %d" % self.max_no_of_address)
        self.start_address = self.address & ~(2 ** self.hostbits - 1)
        self.end_address = self.address | (2 ** self.hostbits - 1)
        print("Address Range :")
        print(" Start Address = ", end = '')
        PrintIP(self.start_address, self.hostbits)
        print(" End Address = ", end = '')
        PrintIP(self.end_address, self.hostbits)
        print("Network ID = ", end = '')
        PrintIP(self.address & ~(2 ** self.hostbits - 1), self.hostbits)
        print("Broadcast Address = ", end = '')
        PrintIP(self.address | (2 ** self.hostbits - 1), self.hostbits)

        self.network_mask = ~(2 ** self.hostbits - 1)
        print("Network Mask = ", end = '')
        PrintIP(self.network_mask, self.hostbits)


    def Subnets(self):
        print("\nEnter the number of subnets in the range(%d, %d)" % (0,
self.max_no_of_subnets ))
        self.no_of_subnets = int(input())
        print("")
        if self.no_of_subnets > self.max_no_of_subnets :
            exit("Error !!! Follow the range")

        subnet_sizes = list()
        print("Enter the number of hosts in each subnet :")
        for i in range(0, self.no_of_subnets) :
            subnet_sizes.append(int(input()) + 2)
            pos = math.ceil(math.log(subnet_sizes[i], 2))
            subnet_sizes[i] = 2**pos

        print("")
        if sum(subnet_sizes) > self.max_no_of_address :
            exit("Subnets cannot accomodate required number hosts")

        print("Subnet Mask, SubnetID and BroadcastID for all the subnets
are as follows :")
        networkid = self.start_address

        for i in range(0, self.no_of_subnets) :
            mask = ~(subnet_sizes[i] - 1)
            print("%d) \tSubnetMask :" % (i+1), end = " ")
            PrintMask(mask)

            print("\tSubnetID :", end = " ")
            PrintIP(networkid, math.log(subnet_sizes[i], 2))
            networkid = networkid + subnet_sizes[i]
            print("\tBroadcastID :", end = " ")
            PrintIP(networkid - 1, math.log(subnet_sizes[i], 2))
            print("\tMaskBits : %d" %(32 - math.log(subnet_sizes[i], 2)))
            print("")
```

```
def PrintMask(address) :
    print((address & 0xFF000000) >> 24, (address & 0xFF0000) >> 16,
(address & 0xFF00) >> 8, address & 0xFF, sep = ".")

def PrintIP(address, hostbits) :
    print((address & 0xFF000000) >> 24, (address & 0xFF0000) >> 16,
(address & 0xFF00) >> 8, address & 0xFF, sep = ".", end = "")
    print("/%d" % (32 - hostbits))

if __name__ == "__main__" :
    print("Enter the IP address in '/' notation")
    temp = input()
    if '/' not in temp :
        exit("Follow the notation")
    x = list(map(str, temp.split(".")))

    if len(x) != 4 :
        exit("Invalid IP")

    x[3], networkbits = x[3].split('/')
    x = list(map(int, x))

    hostbits = 32 - int(networkbits)
    if hostbits < 3 :
        print("No subnets possible")
        exit()
    for i in x :
        if i < 0 or i > 255 :
            exit("Invalid IP")

    address = 0
    address = x[0] << 24
    address = address + (x[1] << 16)
    address = address + (x[2] << 8)
    address = address + x[3]
    ip = IP(address, hostbits)
    ip.Info()
    ip.Subnets()
```

Simple Client-Server Model

The directory **simple_client_server** includes the implementation of a basic client-server model using socket programming in Python. In this setup, the client sends a connection request to the server. If the server accepts the request, it responds by sending back the message **"Hello World!!!"** to the client.

Usage – Run the following commands in two separate terminals:

```
python3 server.py
python3 client.py
```

UDP Communication

The directory **udp** contains the implementation of client-server communication using the User Datagram Protocol (UDP), which operates at the Transport Layer. In this setup, the client requests a file from the server. If the requested file is available, the server sends it back to the client. The client then saves the received file in its current working directory.

Usage – Run the following commands in two separate terminals:

```
python3 udp_server.py
python3 udp_client.py <file_name>
```

TCP Communication

The directory **tcp** contains the implementation of client-server communication using the Transmission Control Protocol (TCP), which is a Transport Layer protocol. In this setup, the client sends a file request to the server. If the requested file is available, the server transmits it back to the client. The client then saves the received file in its current working directory.

Usage – Run the following commands in two separate terminals:

```
python3 tcp_server.py
python3 tcp_client.py <file_name>
```

END OF REPORT